

Final project script 1

Step 1. Localize the centroid of each coin

Your task in this step is to segment the coins from the background in the image and find the centroid of each distinct segmented coin

- Otsu threshold the image to create an initial, but noisy segmentation. Use your 'OtsuThreshold' function we created in the class. Store the result in **msk**.
- Filter the mask to fill all the holes and remove all the false positives. One option is to perform a dilation operation on **msk** to fill in the segmentation. You need to choose a structuring element just large enough to fill in all the holes. If you store the result in **msk_dil** perform an erosion operation on **msk_dil** to separate the coins that were joined together by dilation so that a distinct centroid can be found. Here, you would choose a structuring element large enough to disconnect each coin. Storing the result in **msk_dil_erd** will display the result.
- Use the built-in function `regionprops` to find the centroid of each coin as the centroid of each connected component in your filtered mask. The output is a structure array. The `Centroid` property is a matrix of the closest integer, in [Nx2] matrix **centroid**, where N is the number of components and the columns contain the x-y coordinates for each component. The `Area` property is a vector of the number of pixels) of each component in [Nx1] vector **component_size**.

It is recommended that you write the code in chronological order and run the script each time you create a new assessed or displayed variable. When creating **msk**, and you will see your initial thresholding result displayed in a figure. All assessed variables have been initialized as empty to avoid errors while you are developing your script.

Script ?

[Save](#)[Reset](#)[MATLAB Documentation \(https://www.mathworks.com/help/\)](https://www.mathworks.com/help/)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

```
44 % Define the filter size we will use in step 2:
45 filtsize = 85;
46
47 % Creating test image 'im' by splicing together two built in images.
48 % Also zero-padding (adding zeros around the border) with half the
49 % filter size (filtsize) we will use so that the filter could be
50 % centered on any actual image pixel, including those near the border.
51 % 'coins.png' contains bright nickels and dimes on a dark background
52 % 'eight.tif' contains dark quarters on a bright background, so we invert it
53 % to match 'coins.png'
54 im1 = imread('coins.png');
55 [r,c] = size(im1);
56 im2 = imread('eight.tif');
57 [r2,c2] = size(im2);
58 filtsizeh = floor(filtsize/2);
59 im = zeros(r+r2+filtsize,c+filtsize);
60 im(filtsizeh+1:filtsizeh+r+r2,filtsizeh+1:filtsizeh+c) = [im1;255-im2(:,1:c)];
61 [r,c] = size(im);
```

[▶ Run Script](#)

Assessment: All Tests Passed

[Submit](#)

✓ Is msk correct?

✓ Is centroid correct?

✓ Use regionprops

Output

centroid =

79	149
103	429
98	92
138	188
153	124
165	346
162	251
191	77
217	162
240	477
259	114
289	360
278	215
307	144

component_size =

14x1 struct array with fields:

Area

